المحاضره الرابعة

LECTURE

4&3

DR.AZHAAR A.HUSSAN

STRUCTURE OF A PROGRAM

- A computer program is a sequence of instructions that tell the computer what to do.

- **1-1-2Statements and expressions**
- The most common type of instruction in a program is the **statement**.
- A statement in C++ is the smallest independent unit in the language.
- In C++, we write statements in order to convey to the compiler that we want to perform a task.
- Statements in C++ are terminated by a semicolon.
- There are many different kinds of statements in C.++

- The following are some of the most common types of simple statements:
- int x;
- x = 5
- cout << x;

- An **expression** is a mathematical entity that evaluates to a value.
- For example, in math, the expression 2+3 evaluates to the value 5
- **For example,** the statement x = 2 + 3; is a valid assignment statement.
- The expression 2+3 evaluates to the value of .5This value of 5 is then assigned to x.
-

2-2 L KEYWORDS

- *Keywords* in C++ are explicitly reserved words that have a strict meaning and may not be used in any other way. They include words used for type declarations, such as int, char, and float; words used for statement syntax, such as do, for, and if; and words used for access control, such as public, protected, and private. Table 2.2 shows the keywords in current C++ systems.

,asm, auto, bool, break, case, catch, char, class, const, const_cast
,continue, default, delete, do, double, dynamic_cast, else, enum
,explicit, export, extern, false, float, for, friend, goto, if
,inline, int, long, mutable, namespace, new, operator, private
,protected, public, register, reinterpret_cast, return, short
,signed, sizeof, static, static_cast, struct, switch, template, this
,throw, true, try, typedef, typeid, typename, union, unsigned, using
virtual, void, volatile, wchar_t, while

IDENTIFIERS, AND NAMING THEM

- The name of a variable, function, class, or other entity in C++ is called an **identifier**
- there are a few rules that must be followed when naming identifiers:

1. The identifier cannot be a keyword. Keywords are reserved.
2. The identifier can only be composed of letters, numbers, and the underscore character. That means the name can not contains symbols (except the underscore) nor whitespace.
3. The identifier must begin with a letter or an underscore. It can not start with a number.
4. C++ distinguishes between lower and upper case letters.

- **nValue** is different than **NVALUE.**
- int value; *// correct*
- int Value; *// incorrect (should start with lower case letter(*
- int VALUE; *// incorrect (should start with lower case letter(*
-
- int VaLuE; *// incorrect;*

- If the identifier is a multiword name, there are two common conventions: separated by
- **underscores**, or **intercapped**.
- int my_variable_name; *// correct (separated by underscores(*
- int myVariableName; *// correct (intercapped(*
- int my variable name; *// incorrect (spaces not allowed(*
- int MyVariableName; *// incorrect (should start with lower case letter*

- **Very important:** The C++ language is a "case sensitive" language.
- That means that an identifier written in capital letters is not equivalent to another one with the same name but written in small letters.
- Thus, for example, the **RESULT** variable is not the same as the **result** variable or the **Result** variable.
- These are three different variable identifiers.

VARIABLES2-5 L

- Variables are a way of reserving memory to hold some data and assign names to them so that we 46735 don't have to remember the numbers like and instead we can use the memory location by simply referring to the variable.
- Every variable is mapped to a unique memory address.
- Every variable in C++ can store a value.
- However, the type of value which the variable can store has to be specified by the programmer.

-:C++ supports the following inbuilt data types •

int (to store integer values) There are three types of integer variables.1
,++in C

- **short**, **int** and **long int**
- **;Short a**
- ;int i
- ;25int i=
- ;299long b=
- ;int s,j,k

float (to store decimal values) Floating point data types comes in **.2**
.three sizes, namely float, double and long double

-
- ;float a
- ;4.84float b=
- ;6e2float a=
- ;double sum

(char (to store characters.3

- ;char a
- ;'char name='s

(1or 0 bool (to store Boolean value either .4

- A variable of type bool can store either value true or false and is mostly used in comparisons and logical operations. When converted to integer, true is any non zero .value and false corresponds to zero

.To see what variable declarations look like in action within a program

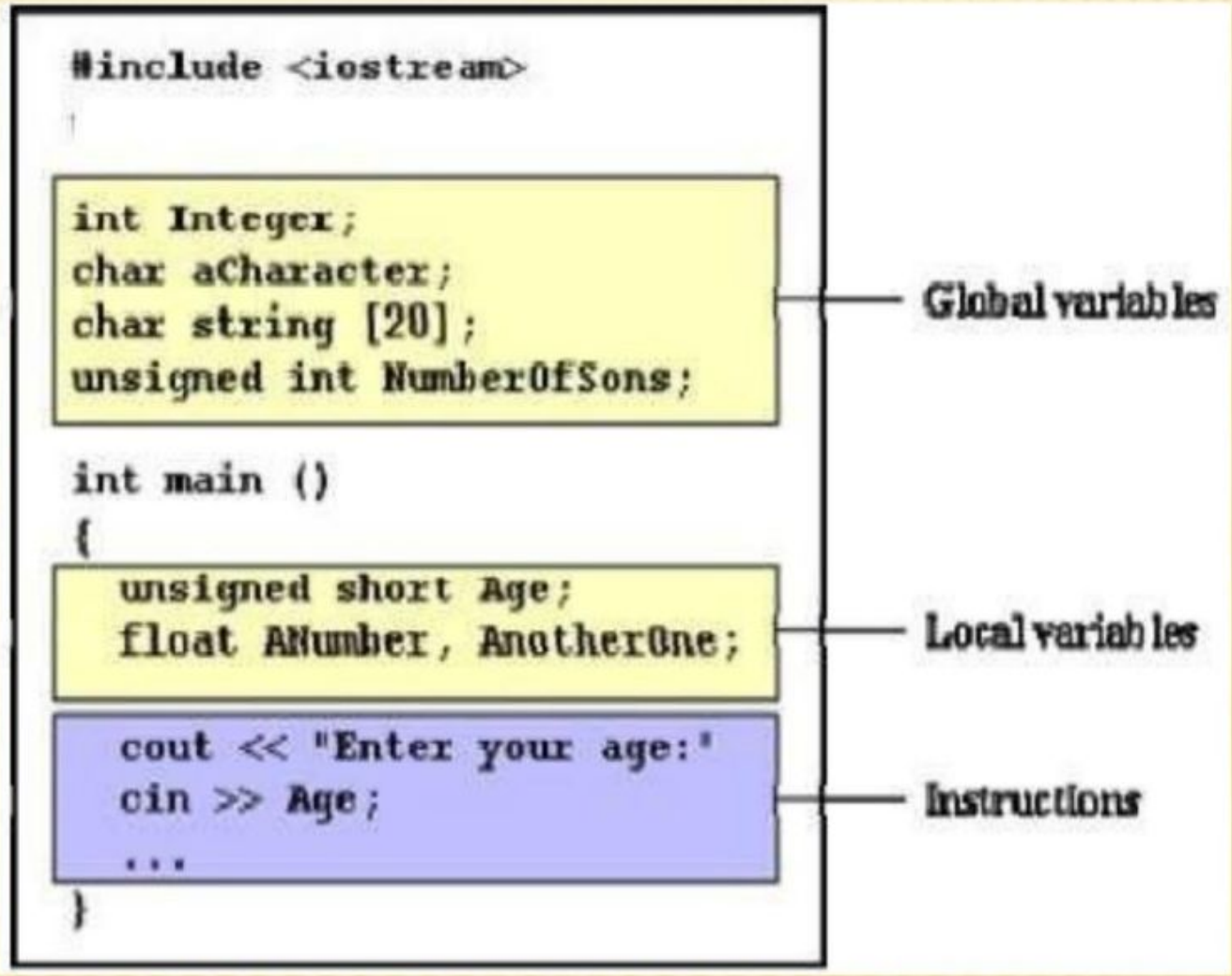
```
operating with variables //
<include <iostream#
() int main
}
:declaring variables //
;int a, b
;int result
:process //
;5a =
;2b =
;1a = a +
;result = a - b
:print out the result //
;"=cout << "result
;cout << result
:terminate the program //
;0return
{
```

SCOPE OF 2-5-1 L VARIABLES

- All the variables that we intend to use in a program must have been declared with its type specifier in an earlier point in the code,
- like we did in the previous code at the beginning of the body of the function main when we declared that a, b, and result were of type int.
- A variable can be either of global or local scope.

1. A global variable is a variable declared in the main body of the source code, outside all functions,
2. while a local variable is one declared within the body of a function or a block.

-


#include <iostream>
int Integer;
char aCharacter;
char string [20];
unsigned int NumberOfSons;
int main ()
{
unsigned short Age;
float ANumber, AnotherOne;
cout << "Enter your age:"
cin >> Age;
...
}
Global variables
Local variables
Instructions

INITIALIZATION OF 2-6 L VARIABLES

- When declaring a regular local variable, its value is by default undetermined.
- But you may want a variable to store a concrete value at the same moment that it is declared.
- In order to do that, you can initialize the variable. There are two ways to do this in C++:
  1. The first one, known as c like, is done by appending an equal sign followed by the value to which the variable will be initialized:

- **; type identifier = initial_value**
- For example
- **;0int a =**

2. The other way to initialize variables, known as constructor initialization, is done by enclosing the initial value between parentheses: (())
   - **; (type identifier (initial_value**
   -
   - For example:
   - **;(0int a (**

- Both ways of initializing variables are valid and equivalent in C++
-
- **//initialization of variables**
- **#include <iostream<**
- **int main()**
- **}**
- **int a=5; // initial value = 5**
- **int b(2); // initial value = 2**
- **int result;// initial value undetermined**
- **a = a + ;3**
- **result = a - b;**
- **cout << result;**
- **return ;0**
- **{**
- 6

5 LECTURE 6&

LITERALS

- Literals are used to express particular values within the source code of a program.
- We have already used these previously to give concrete values to variables or to express messages we wanted our programs to print out, for example, when we wrote:
- a = 5;
- The 5 in this piece of code was a literal constant.
- **Literal constants** can be divided in Integer Numerals, Floating-Point Numerals, Characters, Strings and Boolean Values.

INTEGER NUMERALS

- They are numerical constants that identify integer decimal values.
- Notice that to express a numerical constant we do not have to write quotes (") nor any special character
- **1776**
- **707**
- **273-**

- In addition to decimal numbers ,C++ allows the use as 8literal constants of octal numbers (base) and .(16hexadecimal numbers (base
- to havewe numberoctalan expressto wantwelf .((zero character0 precede it with a
- And in order to express a hexadecimal number we have .(x (zero, x0to precede it with the characters
- all are constantsliteralfollowingtheexample,For :equivalent to each other
- **decimal // 75**
- **octal // 0113**
- **b // hexadecimal4x0**

- (seventy- 75 All of these represent the same number: numeral, octal numeral 10 five) expressed as a base- .and hexadecimal numeral, respectively
- Literal constants, like variables, are considered to have .a specific data type
- By default, integer literals are of type int. However, we can force them to either be unsigned by appending the :u character to it, or long by appending l
  - **int // 75**
  - **u // unsigned int75**
  - **l // long75**
  - **ul // unsigned long75**

- In both cases, the suffix can be specified using either upper or lowercase letters.

FLOATING POINT NUMBERS

- They express numbers with decimals and/or exponents.
- They can include either a decimal point, an e character (that expresses "by ten at the Xth height", where X is an integer value that follows the e character), or both a decimal point and an e character:
  - **3.14159 // 3.14159**
  - **23^10x 6.02 // 23 e6.02**
  - **19^-10x 1.6 // 19 e-1.6**
  - **3.0 // 3.0**

- decimals with numbersvalidfourareThese
.++expressed in C

1. ,The first number is PI
2. ,the second one is the number of Avogadro
3. the third is the electric charge of an electron (an extremely small number) **(all of them (approximated**
4. and the last one is the number three .expressed as a floating point numeric literal

- The default type for floating point literals is .double
- If you explicitly want to express a float or long double numerical literal, you can use the f or l suffixes respectively:
  - **L // long double3.14159**
  - **f // float23e6.02**

- Any of the letters than can be part of a floating-point numerical constant (e, f, l) can be written using either lower or uppercase letters without any difference in their meanings.

CHARACTER AND STRING LITERALS

:There also exist non-numerical constants, like

- 'z'
- 'p'
- "Hello world"
- "?How do you do"
- 'constants'

The first two expressions represent single character constants, and the following two represent string literals composed of several characters.

Notice that to represent a single character we enclose it between single quotes (') and to express a string (which generally consists of more than one character) we enclose it between double quotes (").

- Character and string literals have certain peculiarities, like the escape codes.
- These are special characters that are difficult or impossible to express otherwise in the source code of a program,
- like newline (\n) or tab (\t).
- All of them are preceded by a backslash (\). Here you have a list of some of such escape codes:

- n\ newline
- r\ carriage return
- t\ tab
- b\ backspace
- a\ alert (beep)
- '\ (') single quote
- "\ double quote (")
- ?\ question mark (?)
- \\ (\) backslash

- :For example
- **'n\'**
- **'t\'**
- **"Left \t Right"**
- **"one\ntwo\nthree"**

- String literals can extend to more than a single line of code by putting a backslash sign (\) at the end of each unfinished line. "string “expressed in \ two lines

DEFINED CONSTANTS ((#DEFINE

- You can define your own names for constants that you use very often without having to resort to memory-consuming variables, simply by using the
- :define preprocessor directive. Its format is#
- **define identifier value#**
- :For example
- **3.14159265define PI #**
- **'define NEWLINE '\n#**
- This defines two new constants: PI and NEWLINE. Once they are defined, you can use them in the rest of the code as if they were any other regular constant, for :example

WRITE A PROGRAM TO DETERMINE CIRCUMFERENCE ?OF THE CIRCLE

- defined constants //
- <include <iostream.h#
- 3.14159define PI #
- 'define NEWLINE '\n#
- () int main
- }
- ; // radius5.0double r=
- ;double circle
- ;* PI * r2 circle =
- ;cout << circle
- ;cout << NEWLINE
- { ;0return

31.4159O/P =

- **Notes**
- The #define directive is not a C++ statement but a directive for the preprocessor;
- therefore it assumes the entire line as the directive and does not require a semicolon (;) at its end.
- If you append a semicolon character (;) at the end, it will also be appended in all occurrences within the body of the program that the preprocessor replaces.

(DECLARED CONSTANTS (CONST

- With the const prefix you can declare constants with a specific type in the same way as you would do with a variable:
- **const int pathwidth = 100;**
- **const char tabulator = '\t';**
- Here, path width and tabulator are two typed constants. They are treated just like regular variables except that their values cannot be modified after their definition.